

Manual Agentic Coding Workflow Folder Structure Guide

이 문서는 Manual Agentic Coding Workflow 를 구성하기 위한 폴더 구조와 각 파일의 역할을 설명한다.

목적은 LLM 에게 이 문서를 제공하여 core template → stage template → reusable SKILL → project artifacts 순서로 workflow 를 단계별로 하나씩 만들게 하는 것이다.

1. Core Principle

이 workflow 의 핵심 원칙은 다음과 같다.

Workflow template 은 설계 기준이다.

Reusable SKILL 은 agent 가 실제 실행하는 절차 문서이다.

Project artifact 는 특정 프로젝트에서 생성된 결과물이다.

따라서 폴더 구조는 다음 세 층을 분리해야 한다.

1. Workflow Template Layer

→ SKILL 을 설계하기 위한 기준 문서

2. Reusable Skill Layer

→ 여러 프로젝트에서 반복 실행할 수 있는 SKILL.md

3. Project Execution Layer

→ 실제 프로젝트별 입력, 산출물, 승인 기록, context 파일

중요한 구분은 다음과 같다.

Stage-Specific Skill Template ≠ Reusable Stage SKILL.md

Reusable Stage SKILL.md ≠ Project Artifact

Project Artifact ≠ Approved Decision unless explicitly approved

2. Recommended Top-Level Structure

권장하는 최상위 구조는 다음과 같다.

/workflow_templates

 /core

 /stage_templates

 /specializations

 /tool_wrappers

/skills

- /00_project_intake
- /01_service_goal_definition
- /02_stakeholder_risk_framing
- /03_requirements_acceptance
- /04_domain_modeling
- /05_architecture_contracts
- /06_data_design
- /07_mvp_release_slicing
- /08_test_strategy_validation
- /09_task_breakdown
- /10_implementation_prompt_writing
- /11_tdd_implementation_loop
- /12_review_release_handoff
- /13_workflow_retrospective

- /workflow
 - /00_intake
 - /01_goal
 - /02_stakeholders_risk
 - /03_requirements
 - /04_domain
 - /05_architecture
 - /06_data
 - /07_mvp_release
 - /08_test_strategy
 - /09_tasks
 - /10_prompts
 - /11_implementation_results
 - /12_review_release_handoff
 - /13_retrospective
- /context

각 최상위 폴더의 의미는 다음과 같다.

Folder	Purpose	직접 실행 여부
/workflow_templates	SKILL 을 만들기 위한 설계 기준	실행하지 않음
/skills	agent 가 실제 실행하는 reusable SKILL.md 모음	실행함
/workflow	특정 프로젝트에서 생성되는 산출물과 context	실행 결과

3. /workflow_templates Layer

/workflow_templates 는 실행용 폴더가 아니다.
이 폴더는 reusable SKILL.md 를 만들기 위한 설계 기준을 보관한다.

```
/workflow_templates
/core
  core_skill_template.md

/stage_templates
  00_project_intake_skill_template.md
  01_service_goal_skill_template.md
  02_stakeholder_risk_skill_template.md
  03_requirements_skill_template.md
  04_domain_modeling_skill_template.md
  05_architecture_skill_template.md
  06_data_design_skill_template.md
  07_mvp_release_skill_template.md
  08_test_strategy_skill_template.md
  09_task_breakdown_skill_template.md
  10_implementation_prompt_skill_template.md
  11_implementation_loop_skill_template.md
  12_review_release_skill_template.md
  13_retrospective_skill_template.md

/specializations
  web_saas.md
  internal_tool.md
  mobile_app.md
  ai_data_product.md
  regulated_security_sensitive.md
  brownfield_legacy.md

/tool_wrappers
  claude_code_wrapper.md
  codex_wrapper.md
  antigravity_wrapper.md
```

4. /workflow_templates/core

Purpose

/workflow_templates/core 는 모든 SKILL 이 반드시 따라야 하는 최소 공통 규칙을 보관한다.

대표 파일:

/workflow_templates/core/core_skill_template.md

이 파일은 모든 SKILL 의 공통 운영 계약을 정의한다.

포함해야 하는 내용:

- agent role and operating mode
- input precedence rules
- USER_DIRECTIVES.md handling
- artifact reading rules
- approved decision / assumption / open question separation
- output artifact rules
- context_packet.md update rules
- human approval gate
- failure handling
- do-not-do rules

포함하지 않아야 하는 내용:

- requirements 작성 세부 방법
- DDD aggregate 설계 방법
- API contract 설계 방법
- database schema 설계 방법
- TDD implementation loop 세부 절차
- security review checklist 전체
- project-type-specific rules
- tool-specific command conventions

이런 내용은 stage template, specialization addendum, tool wrapper 로 분리한다.

5. /workflow_templates/stage_templates

Purpose

/workflow_templates/stage_templates 는 각 stage 의 reusable SKILL.md 를 만들기 위한 stage 별 설계 템플릿을 보관한다.

예시:

/workflow_templates/stage_templates/03_requirements_skill_template.md

이 파일은 agent 가 직접 실행하는 파일이 아니다.

이 파일은 /skills/03_requirements_acceptance/SKILL.md 를 만들 때 사용하는 설계 기준이다.

Stage-specific skill template 은 각 stage 에 대해 다음을 정의한다.

- Stage Purpose
- Core Question
- Always Read Inputs

- Read If Applicable Inputs
- Do Not Read By Default
- Missing Input Handling
- Mandatory Artifacts
- Conditional Artifacts
- N/A Record Rules
- Required Output Structure
- Traceability Requirements
- Stage-Specific Validation Checklist
- Stage-Specific Human Approval Gate
- Next context_packet.md Rules

Example

03_requirements_skill_template.md 는 다음을 정의한다.

Stage Purpose:

- service goal 을 functional / non-functional requirements 로 분해한다.
- 각 requirement 를 acceptance criteria 와 연결한다.
- requirements 가 testable 한지 점검한다.

Always Read Inputs:

- /workflow/context/context_packet.md
- /workflow/context/DECISIONS.md
- /workflow/01_goal/01_service_goal.md
- /workflow/02_stakeholders_risk/02_stakeholders.md

Mandatory Artifacts:

- /workflow/03_requirements/03_requirements.md
- /workflow/03_requirements/03_acceptance_criteria.md
- /workflow/03_requirements/result.md

6. /workflow_templates/specializations

Purpose

/workflow_templates/specializations 는 project type 에 따라 추가해야 하는 규칙을 보 관한다.

예시:

- /workflow_templates/specializations/web_saas.md
- /workflow_templates/specializations/mobile_app.md
- /workflow_templates/specializations/ai_data_product.md
- /workflow_templates/specializations/regulated_security_sensitive.md
- /workflow_templates/specializations/brownfield_legacy.md

Specialization addendum 은 stage-specific SKILL 을 대체하지 않는다.
필요한 경우 core + stage-specific SKILL 위에 추가로 적용된다.

Specialization 이 추가할 수 있는 내용:

- additional inputs
- additional questions
- additional artifacts
- additional validation needs
- additional risks
- additional approval requirements

예시:

ai_data_product.md

→ dataset provenance, labeling policy, evaluation metrics, model risk, reproducibility 요구 추가

regulated_security_sensitive.md

→ threat model, audit log, privacy impact, release blocker 기준 추가

mobile_app.md

→ device permission, offline sync, app store release, platform-specific validation 추가

7. /workflow_templates/tool_wrappers

Purpose

/workflow_templates/tool_wrappers 는 Claude Code, Codex, Antigravity 등 agentic coding tool 별 실행 환경 차이를 다룬다.

예시:

/workflow_templates/tool_wrappers/claude_code_wrapper.md

/workflow_templates/tool_wrappers/codex_wrapper.md

/workflow_templates/tool_wrappers/antigravity_wrapper.md

Tool wrapper 가 다루는 것:

- skill file location
- command invocation convention
- artifact output location
- sandbox / permission rules
- review UI convention
- tool-specific execution notes

Tool wrapper 가 다루지 말아야 하는 것:

- domain modeling decisions
- requirement decisions

- architecture decisions
- data design decisions
- test strategy decisions
- security policy decisions

Tool wrapper 는 실행 환경 차이만 처리한다.

8. /skills Layer

Purpose

/skills 는 agent 가 실제 실행하는 reusable SKILL.md 를 보관한다.

각 SKILL 은 별도의 폴더를 가지고, 폴더 안의 파일 이름은 항상 SKILL.md 로 둔다.

권장 구조:

/skills

 /00_project_intake
 SKILL.md

 /01_service_goal_definition
 SKILL.md

 /02_stakeholder_risk_framing
 SKILL.md

 /03_requirements_acceptance
 SKILL.md

 /04_domain_modeling
 SKILL.md

 /05_architecture_contracts
 SKILL.md

 /06_data_design
 SKILL.md

 /07_mvp_release_slicing
 SKILL.md

 /08_test_strategy_validation
 SKILL.md

 /09_task_breakdown

SKILL.md

/10_implementation_prompt_writing
SKILL.md

/11_tdd_implementation_loop
SKILL.md

/12_review_release_handoff
SKILL.md

/13_workflow_retrospective
SKILL.md

Why Folder-Based SKILLS?

실행용 SKILL 은 skill-name.md 보다 skill-name/SKILL.md 방식이 좋다.

이유:

- SKILL 별 보조 파일을 둘 수 있다.
- example input/output 을 추가할 수 있다.
- artifact contract 를 별도 파일로 분리할 수 있다.
- checklist, templates, addendum 을 함께 관리할 수 있다.
- 나중에 하나의 stage 를 여러 SKILL 로 분할하기 쉽다.
- tool-specific wrapper 나 specialization 과 연결하기 쉽다.

예시:

/skills/08_test_strategy_validation
SKILL.md
artifact_contract.yml
example_inputs.md
example_outputs.md
validation_checklist.md

처음에는 SKILL.md 하나만 두어도 된다.
필요해질 때 보조 파일을 추가한다.

9. Relationship Between Stage Template and Reusable SKILL

Stage template 과 reusable SKILL 의 관계는 다음과 같다.

core_skill_template.md
+ stage-specific skill template
+ optional specialization addendum

+ optional tool wrapper
→ reusable SKILL.md

예시:

/workflow_templates/core/core_skill_template.md
+ /workflow_templates/stage_templates/O3_requirements_skill_template.md
+ /workflow_templates/specializations/web_saas.md, if applicable
→ /skills/O3_requirements_acceptance/SKILL.md

구분:

File	Role	Executed by agent?
core_skill_template.md	모든 SKILL 의 공통 규칙	No
O3_requirements_skill_template.md	Stage3 SKILL 설계 기준	No
web_saas.md	Web SaaS 추가 규칙	No, applied as addendum
/skills/O3_requirements_acceptance/SKILL.md	실제 실행용 SKILL	Yes

10. /workflow Project Execution Layer

Purpose

/workflow 는 특정 프로젝트에서 실제 SKILL 을 실행한 결과를 저장한다.

이 폴더는 reusable SKILL library 가 아니다.

프로젝트별 artifact, context, decisions, approval 기록을 저장하는 실행 결과 폴더이다.

권장 구조:

/workflow
 /00_intake
 00_project_intake.md
 00_existing_context_review.md
 result.md
 USER_DIRECTIVES.md
 review_notes.md

 /01_goal
 01_service_goal.md
 result.md
 USER_DIRECTIVES.md
 review_notes.md

 /02_stakeholders_risk
 02_stakeholders.md
 02_risk_privacy_screening.md

result.md

/03_requirements

03_requirements.md

03_acceptance_criteria.md

03_traceability_matrix.md

result.md

/04_domain

04_ubiquitous_language.md

04_domain_model.md

04_bounded_contexts.md

04_business_rules_invariants.md

04_domain_events.md

result.md

/05_architecture

05_architecture_plan.md

05_module_boundaries.md

05_api_contracts.md

05_integration_contracts.md

05_architecture_decisions.md

result.md

/06_data

06_conceptual_data_model.md

06_logical_schema.md

06_physical_schema.md

06_indexes.md

06_migration_plan.md

06_data_security_rules.md

result.md

/context

artifact_manifest.yml

context_packet.md

DECISIONS.md

ASSUMPTIONS.md

OPEN_QUESTIONS.md

REJECTED_OPTIONS.md

TRACEABILITY_MATRIX.md

APPROVAL_LOG.md

11. /workflow/context

Purpose

/workflow/context 는 stage 간 context handoff 와 승인 상태를 관리한다.

/workflow/context
artifact_manifest.yml
context_packet.md
DECISIONS.md
ASSUMPTIONS.md
OPEN_QUESTIONS.md
REJECTED_OPTIONS.md
TRACEABILITY_MATRIX.md
APPROVAL_LOG.md

각 파일의 역할은 다음과 같다.

File	Role
artifact_manifest.yml	artifact 의 존재, 상태, 승인 여부, superseded 여부 관리
context_packet.md	다음 stage 에 필요한 최소 operational context
DECISIONS.md	human-approved decisions only
ASSUMPTIONS.md	아직 검증되지 않은 working assumptions
OPEN_QUESTIONS.md	unresolved questions
REJECTED_OPTIONS.md	다시 제안하지 말아야 할 rejected or superseded options
TRACEABILITY_MATRIX.md	goal → requirement → test → task → evidence 연결
APPROVAL_LOG.md	human approval 기록

중요 원칙:

context_packet.md is a navigation layer, not the sole source of truth.

즉, 중요한 결정은 DECISIONS.md, APPROVAL_LOG.md, 승인된 stage artifact 로 확인해야 한다.

12. Stage Artifact Contract vs SKILL Split

각 stage 는 먼저 외부 artifact contract 를 가져야 한다.

Stage Artifact Contract = Mandatory Artifacts + Conditional Artifacts + N/A Record + Next Context Rules

중요 원칙:

Downstream stage should depend on approved artifacts,
not on how many SKILLS produced them.

즉, 하나의 stage 를 하나의 SKILL 로 만들든 여러 개의 SKILL 로 분할하든, 다음 stage 는 SKILL 개수가 아니라 승인된 artifact 를 기준으로 연결되어야 한다.

예시:

Stage 8 Test Strategy official artifacts:

- 08_test_strategy.md
- 08_acceptance_tests.md
- 08_validation_commands.md
- 08_manual_test_plan.md

Stage 8 이 하나의 SKILL 로 실행되어도 다음과 같다.

/skills/08_test_strategy_validation/SKILL.md

→ 위 4 개 artifact 생성

Stage 8 이 여러 SKILL 로 분할되어도 다음과 같다.

/skills/08a_test_scope/SKILL.md

/skills/08b_acceptance_tests/SKILL.md

/skills/08c_validation_commands/SKILL.md

/skills/08d_manual_test_plan/SKILL.md

/skills/08e_test_strategy_finalizer/SKILL.md

→ 최종적으로 동일한 공식 artifact set 생성

Stage 9 Task Breakdown 은 Stage 8 의 내부 SKILL 개수가 아니라 다음 artifact 를 읽는다.

/workflow/08_test_strategy/08_test_strategy.md

/workflow/08_test_strategy/08_acceptance_tests.md

/workflow/08_test_strategy/08_validation_commands.md

/workflow/08_test_strategy/08_manual_test_plan.md

13. Recommended Workflow Creation Order for LLM

LLM 에게 workflow 를 단계별로 만들게 할 때는 다음 순서가 가장 안정적이다.

- Step 1. Create or verify core_skill_template.md
- Step 2. Create stage artifact contracts for Stage 0~13
- Step 3. Create stage-specific skill templates for Stage 0~13
- Step 4. Create 1-stage-1-skill reusable SKILL.md files
- Step 5. Split large stages into smaller SKILLS only when needed
- Step 6. Create project-type specialization addenda
- Step 7. Create tool-specific wrappers
- Step 8. Apply the skills to a small pilot project

Step 9. Record agent failure patterns
Step 10. Improve templates and skills iteratively

처음부터 모든 stage 를 세분화하지 않는다.
먼저 1-stage-1-skill 버전을 만들고, 너무 큰 stage 만 나중에 분할한다.

14. LLM Prompt: Create Stage Templates One by One

다음 프롬프트를 사용하여 LLM 에게 stage-specific skill template 을 하나씩 만들게 할 수 있다.

You are designing a Stage-Specific Skill Template for a Manual Agentic Coding Workflow.

Use the following source documents:

- /workflow_templates/core/core_skill_template.md
- /workflow_templates/stage_templates/[[previous stage templates if relevant]]
- agentic-coding-workflow-concept-and-design.md
- skill-template-design-principles.md

Target stage:

- [[stage number and stage name]]

Create a stage-specific skill template, not an executable SKILL.md yet.

The template must define:

1. Stage Purpose
2. Core Question
3. Always Read Inputs
4. Read If Applicable Inputs
5. Do Not Read By Default
6. Missing Input Handling
7. Mandatory Artifacts
8. Conditional Artifacts
9. N/A Record Rules
10. Required Output Structure
11. Traceability Requirements
12. Stage-Specific Validation Checklist
13. Stage-Specific Human Approval Gate
14. Next context_packet.md Rules

Rules:

- Do not repeat the entire core_skill_template.md.
- Define only stage-specific extensions.
- Do not include project-specific content.
- Assume a greenfield MVP production project as the default baseline.

- Leave hooks for specialization addenda.
 - Make the template context-reset tolerant.
-

15. LLM Prompt: Create Reusable SKILL.md from Template

Stage-specific template 을 만든 뒤, 다음 프롬프트로 실행용 SKILL 을 만들게 한다.

You are creating an executable reusable SKILL.md for a Manual Agentic Coding Workflow.

Use the following source documents:

- /workflow_templates/core/core_skill_template.md
- /workflow_templates/stage_templates/[[stage]]_skill_template.md
- /workflow_templates/specializations/[[optional specialization]].md, if applicable

Target output:

- /skills/[[stage_skill_name]]/SKILL.md

Create a reusable SKILL.md that an agent can actually execute.

The SKILL.md must:

1. Follow the core skill template.
2. Implement the stage-specific template.
3. Define required inputs and conditional inputs.
4. Define output artifacts and required sections.
5. Define the execution procedure.
6. Define traceability rules.
7. Define decision / assumption / open question handling.
8. Define context_packet.md update rules.
9. Include a human approval gate.
10. Avoid project-specific details.

Rules:

- This SKILL.md is reusable across projects.
 - Project-specific information must come from approved artifacts, context_packet.md, artifact_manifest.yml, and U
 - Do not assume missing information.
 - Mark assumptions explicitly.
 - Do not treat agent proposals as approved decisions.
 - The final file must be saved as SKILL.md inside its own folder.
-

16. LLM Prompt: Split a Large Stage SKILL

큰 stage 를 나누어야 할 때는 다음 프롬프트를 사용한다.

The following stage SKILL is too large and should be split into smaller SKILLS.

Input files:

- /skills/[[large_stage_skill]]/SKILL.md
- /workflow_templates/stage_templates/[[stage]]_skill_template.md
- /workflow_templates/core/core_skill_template.md

Split the stage into smaller reusable SKILLS.

Rules:

1. Do not change the official Stage Artifact Contract unless necessary.
2. Downstream stages must continue to depend on approved stage artifacts, not on internal SKILL names.
3. Each smaller SKILL must have a clear purpose and output responsibility.
4. If intermediate artifacts are created, mark them as internal working artifacts unless they are promoted to official artifacts.
5. Include a finalizer SKILL if needed to consolidate artifacts, update context_packet.md, and prepare the human artifact.
6. Update artifact_manifest expectations if official artifacts change.

Return:

- proposed split structure
 - purpose of each smaller SKILL
 - official artifacts preserved
 - internal working artifacts, if any
 - downstream impact analysis
-

17. Naming Conventions

Template Files

Use snake_case and include _skill_template.md.

00_project_intake_skill_template.md
03_requirements_skill_template.md
08_test_strategy_skill_template.md

Reusable SKILL Folders

Use stage number + stable skill name.

/skills/03_requirements_acceptance/SKILL.md
/skills/04_domain_modeling/SKILL.md
/skills/08_test_strategy_validation/SKILL.md

Project Artifact Files

Use stage number + artifact purpose.

03_requirements.md
03_acceptance_criteria.md
05_architecture_plan.md
08_validation_commands.md

Context Files

Use stable uppercase names for persistent workflow context.

DECISIONS.md
ASSUMPTIONS.md
OPEN_QUESTIONS.md
REJECTED_OPTIONS.md
TRACEABILITY_MATRIX.md
APPROVAL_LOG.md

18. Final Rule

The most important structural rule is this:

Templates define how to create SKILLS.
SKILLS define how agents produce artifacts.
Artifacts define what downstream stages can trust.
Approvals define what becomes source of truth.

Therefore:

Do not make downstream stages depend on prompt history.
Do not make downstream stages depend on internal SKILL split.
Do not make context_packet.md the only source of truth.
Do not treat unapproved artifacts as final decisions.

The workflow should remain:

Reusable
Inspectable
Artifact-first
Human-approved
Traceable
Context-reset tolerant
Production-aware